
CS 301

High-Performance Computing

Assignment 05 - Particle Insertion, Deletion --- and Parallel Mover Scalability ---

Dhruvi Jobanputra (202301401)
Prasham Shah (202301252)

March 2026

Contents

1	Introduction	3
2	Problem Description	3
3	Implementation Approach	3
3.1	Cloud-In-Cell Interpolation	4
3.2	Serial Particle Movers	4
3.3	Parallel Particle Movers	4
4	Experimental Setup	5
4.1	Systems Description	5
4.2	Experiment Definitions	5
4.2.1	Experiment 1: Scaling of Execution Time with Particle Count	5
4.2.2	Experiment 2: Parallel Implementation and Scalability	5
4.2.3	Per-Particle Execution Time vs PPC	6
5	Results	7
5.1	Experiment 1: Scaling with Particle Count	7
5.2	Per-Particle Execution Time vs PPC	9
5.3	Experiment 2: Parallel Scalability	11
5.4	Comparison with Assignment 04 (Insertion vs No Insertion)	17
5.5	Discussion of Observed Trends	20
6	Analysis Tasks	21
7	Conclusion	22

1 Introduction

Particle-mesh methods are widely used in scientific computing problems such as plasma physics, molecular dynamics, and fluid simulation. These methods store the physical state in a particle representation while solving the governing equations on a structured mesh. A common operation in such workflows is interpolation, where particle information is transferred to the mesh.

In this assignment, the particle-mesh code is extended to support particle insertion and deletion. Particles are allowed to move out of the unit square domain, deleted logically, and replaced by newly initialized random particles so that the total particle count remains constant. The performance of this extended mover is studied for serial and parallel implementations on both a laboratory workstation and an HPC cluster.

The study focuses on two main experiments: scaling with particle count in the serial mover and scalability with thread count in the parallel mover. In addition, the per-particle execution time is analyzed as a function of particles per cell (PPC).

2 Problem Description

Consider a unit square domain containing randomly distributed particles with coordinates (x_i, y_i) . The domain is discretized using a structured grid of size $N_X \times N_Y$. The objective is to perform the following operations:

- Interpolate particle contributions onto the grid using the Cloud-In-Cell (CIC) bilinear scheme.
- Update particle positions using a stochastic mover with random displacements bounded by local grid spacing.
- Delete particles that leave the domain and reinsert new random particles to keep the particle count fixed.
- Evaluate the scaling behavior of the serial mover with particle count.
- Evaluate the scalability of the parallel mover with thread count.
- Compare performance on a Lab PC and an HPC cluster.

Mover Variants used in the code:

- **Serial Immediate:** Replace invalid particles immediately.
- **Serial Deferred:** Compact valid particles and refill later.
- **Parallel Immediate:** Thread-parallel immediate replacement.
- **Parallel Deferred:** Thread-parallel compaction and refill.

3 Implementation Approach

The particle-mesh workflow consists of interpolation followed by the mover step. Data layout, memory access, and loop organization are important for obtaining good cache utilization and predictable performance.

3.1 Cloud-In-Cell Interpolation

The structured grid is stored in a one-dimensional array in row-major order. A grid node at logical index (i, j) is accessed as:

$$\text{mesh}[j \cdot \text{GRID_X} + i]$$

Each particle located inside a cell contributes to the four surrounding nodes using bilinear weights. Since each particle performs a constant amount of work, the interpolation runtime is expected to scale linearly with the number of particles.

3.2 Serial Particle Movers

The serial mover updates each particle by applying a random displacement bounded by the grid spacing:

$$x' = x + \delta_x, \quad y' = y + \delta_y$$

where

$$\delta_x \in [-\Delta x, \Delta x], \quad \delta_y \in [-\Delta y, \Delta y]$$

If the new position leaves the domain, the particle is either replaced immediately (Serial Immediate) or marked as invalid and compacted later (Serial Deferred).

In the deferred version, the implementation scans all particles once, keeps valid particles at the front of the array, and refills the tail with newly generated random particles. This makes the array compact and preserves the total particle count.

3.3 Parallel Particle Movers

The parallel mover is implemented using OpenMP. Two versions are used:

- **Parallel Immediate:** each thread updates its own particles independently and replaces invalid particles in place.
- **Parallel Deferred:** each thread compacts valid particles locally, the thread-local counts are converted into prefix-sum offsets, and the compacted blocks are copied back without write conflicts.

A naive shared write-index approach would create a race condition because multiple threads would try to place valid particles into the same destination index. The chunked, per-thread buffer approach avoids this by ensuring that each thread writes to its own region, followed by a small offset computation across threads.

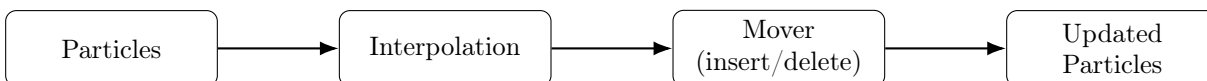


Figure 1: Particle-mesh workflow used in Assignment 05.

4 Experimental Setup

All experiments were conducted using the same particle-mesh code on two different computing environments: a laboratory workstation and an HPC cluster node. Execution time was measured separately for the interpolation phase and the mover phase.

4.1 Systems Description

Lab PC:

- Desktop workstation with multi-core CPU.
- Higher single-core clock frequency.
- Lower memory latency.
- OpenMP thread counts used: 1 for serial runs, 2/4/8/16 for parallel runs.

HPC Cluster Node:

- Multi-core server processor.
- Higher memory bandwidth and larger shared memory hierarchy.
- Lower per-core frequency than the desktop.
- OpenMP thread counts used: 1 for serial runs, 2/4/8/16 for parallel runs.

4.2 Experiment Definitions

4.2.1 Experiment 1: Scaling of Execution Time with Particle Count

The serial mover with insertion and deletion was studied for three grid sizes and five particle counts.

Grid size	Particle counts
250×100	$10^2, 10^4, 10^6, 10^8, 10^9$
500×200	$10^2, 10^4, 10^6, 10^8, 10^9$
1000×400	$10^2, 10^4, 10^6, 10^8, 10^9$

For each configuration, the simulation was executed for `Maxiter` = 10 iterations. The plots show total execution time on a log-log scale.

4.2.2 Experiment 2: Parallel Implementation and Scalability

The parallel mover was studied with a fixed particle count of 14 million and `Maxiter` = 10. Thread counts of 2, 4, 8, and 16 were used.

Parameter	Value
Particles	14 million
Grid sizes	$250 \times 100, 500 \times 200, 1000 \times 400$
Threads	2, 4, 8, 16
Modes compared	Parallel Immediate and Parallel Deferred

The speedup plots compare the parallel mover with insertion/deletion against the baseline parallel mover without insertion/deletion from Assignment 04.

4.2.3 Per-Particle Execution Time vs PPC

The per-particle execution time is plotted as a function of particles per cell (PPC):

$$\text{PPC} = \frac{N_p}{N_X N_Y}$$

This plot is used to analyze how the amount of work per cell affects the execution time per particle.

5 Results

This section contains placeholders for the final plots. Replace the placeholders with the generated figures from the `plots/` folder.

5.1 Experiment 1: Scaling with Particle Count

The required plots are the total execution time versus particle count for each grid size. Each plot compares Lab PC and HPC cluster results.



Figure 2: Experiment 1: Total execution time vs particle count for the 250×100 grid, comparing Lab PC and HPC cluster.

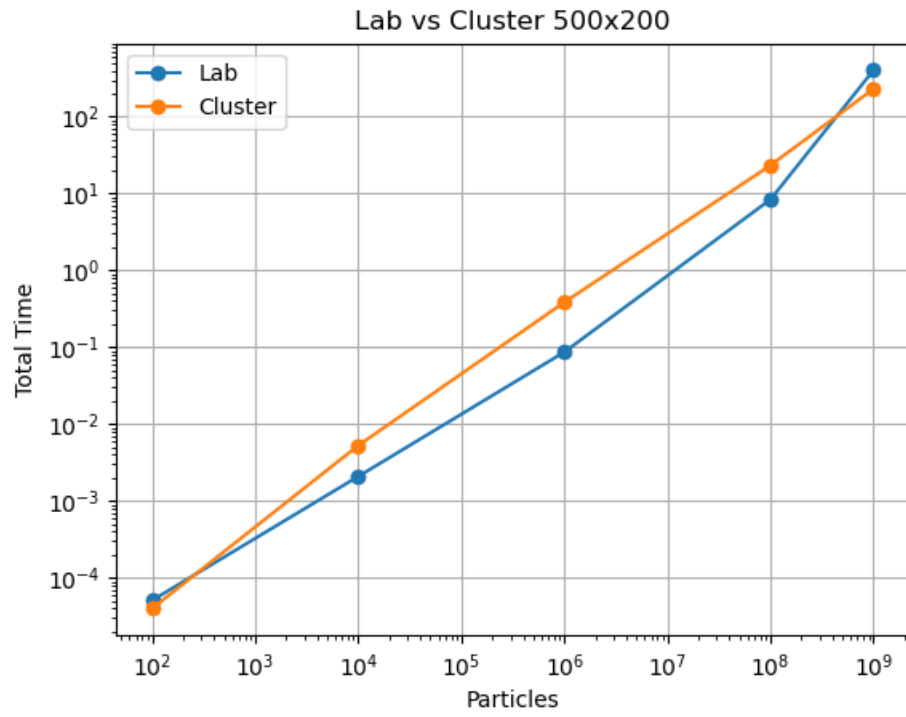


Figure 3: Experiment 1: Total execution time vs particle count for the 500×200 grid, comparing Lab PC and HPC cluster.

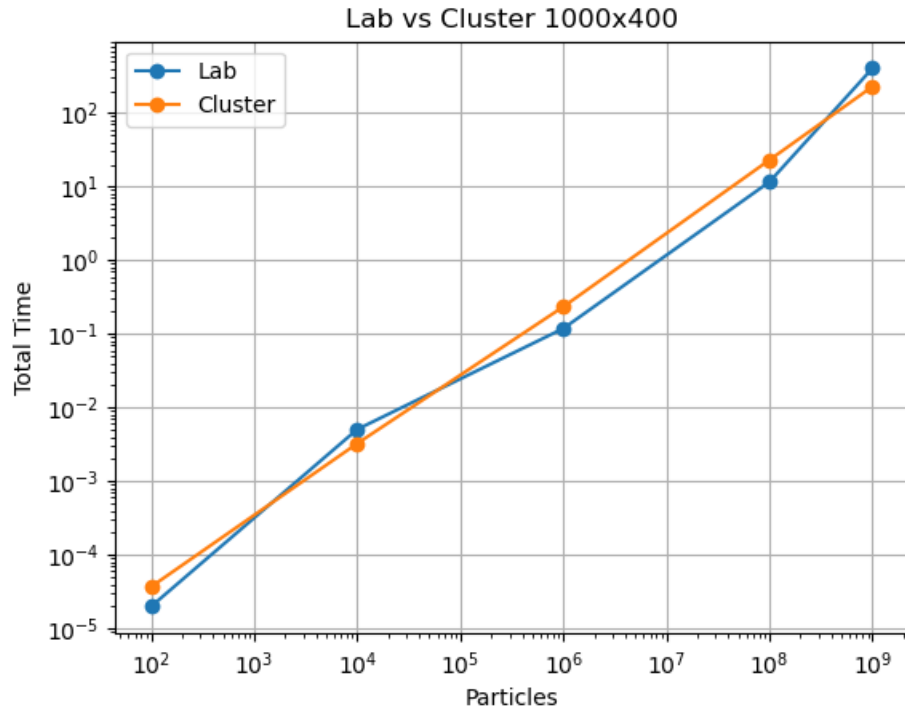


Figure 4: Experiment 1: Total execution time vs particle count for the 1000×400 grid, comparing Lab PC and HPC cluster.

5.2 Per-Particle Execution Time vs PPC

The manual asks for the per-particle execution time as a function of PPC. This helps explain how work per cell affects the cost per particle.

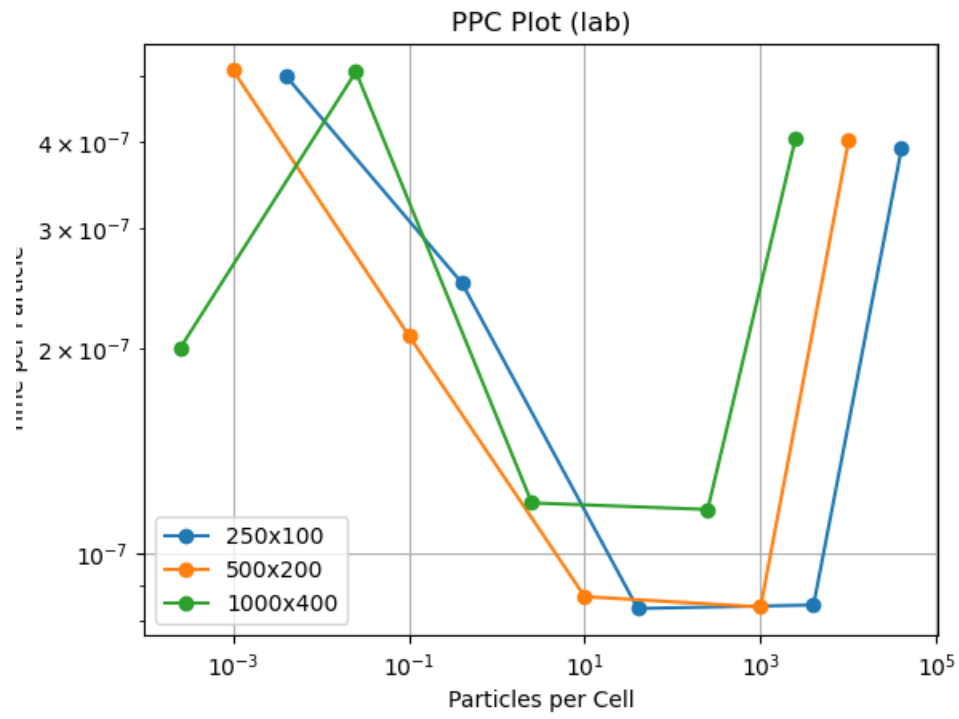


Figure 5: Per-particle execution time vs PPC on the Lab PC.

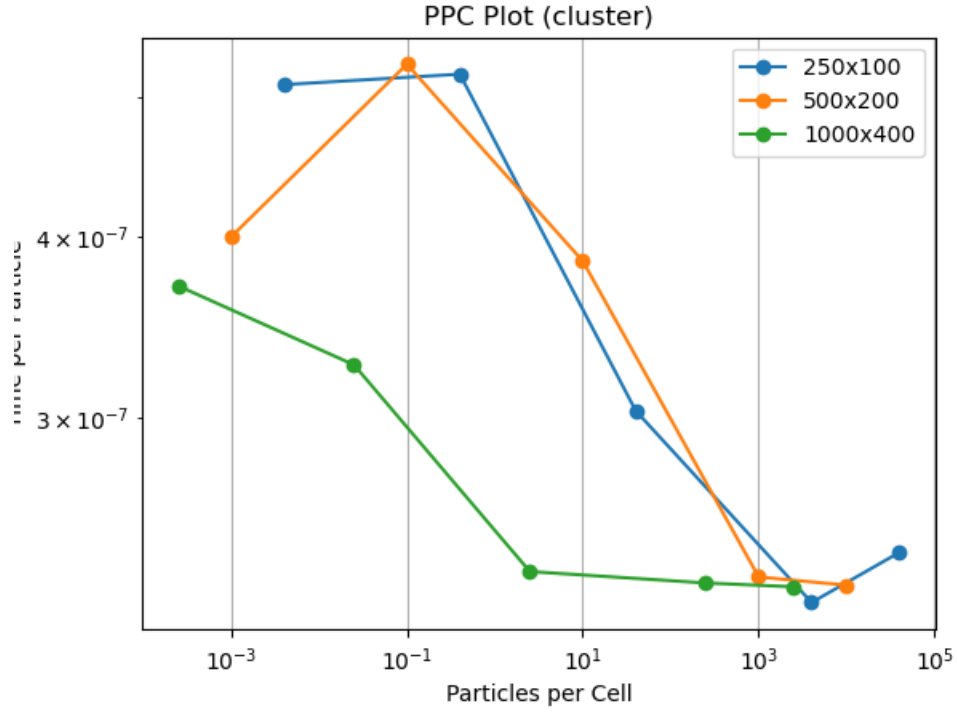


Figure 6: Per-particle execution time vs PPC on the HPC cluster.

5.3 Experiment 2: Parallel Scalability

The required plots show speedup versus number of threads. The final version should include both curves in each figure: the current assignment (Parallel Deferred (with insertion/deletion) and the Assignment 04 baseline (Parallel Immediate (without insertion/deletion)).

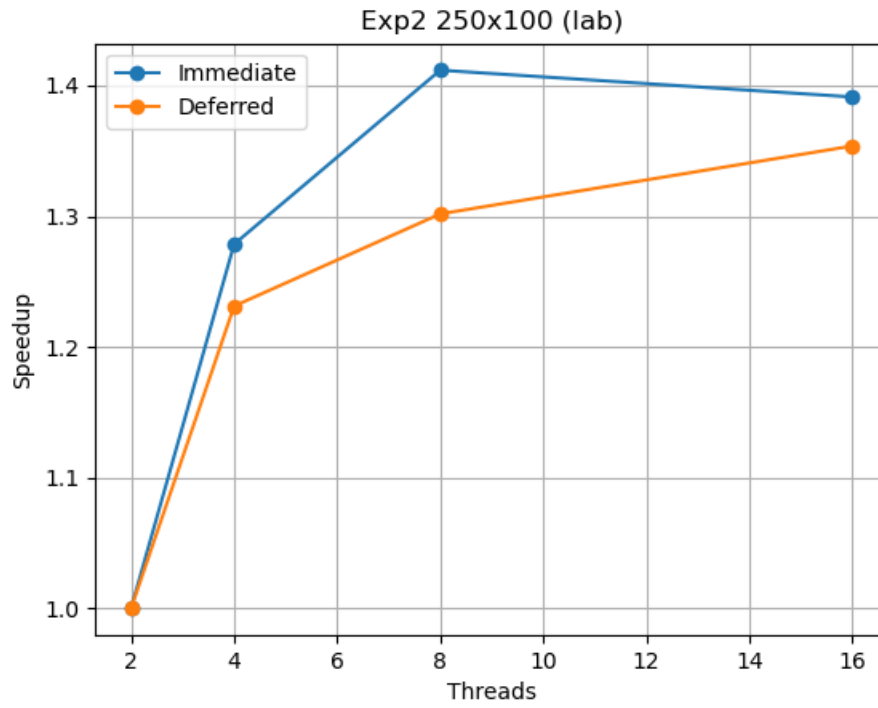


Figure 7: Experiment 2: Speedup vs threads for the 250×100 grid on the Lab PC. The plot includes both Parallel Immediate and Parallel Deferred implementations.

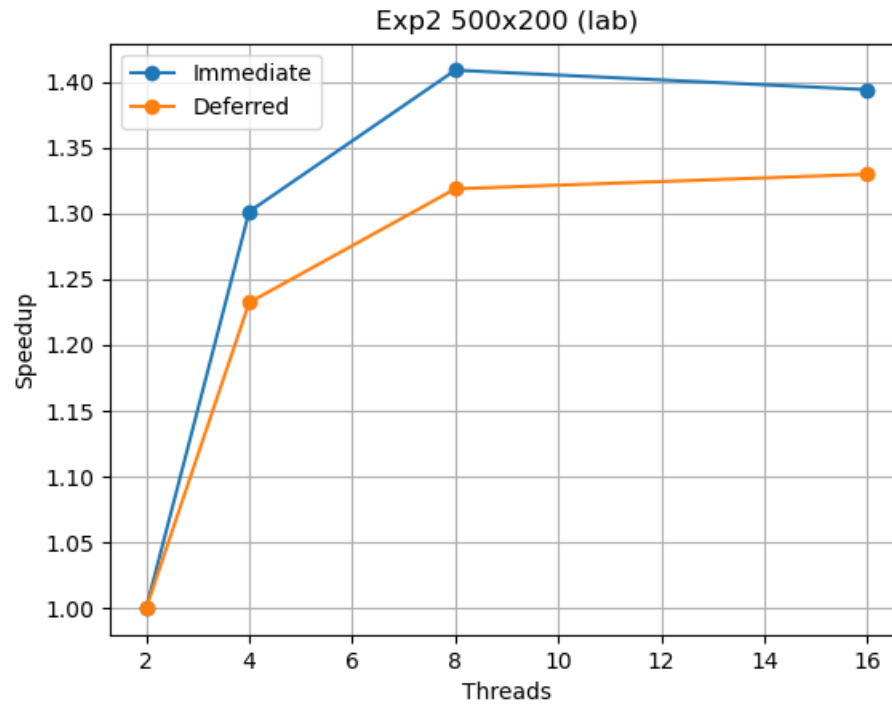


Figure 8: Experiment 2: Speedup vs threads for the 500×200 grid on the Lab PC. The plot includes both Parallel Immediate and Parallel Deferred implementations.

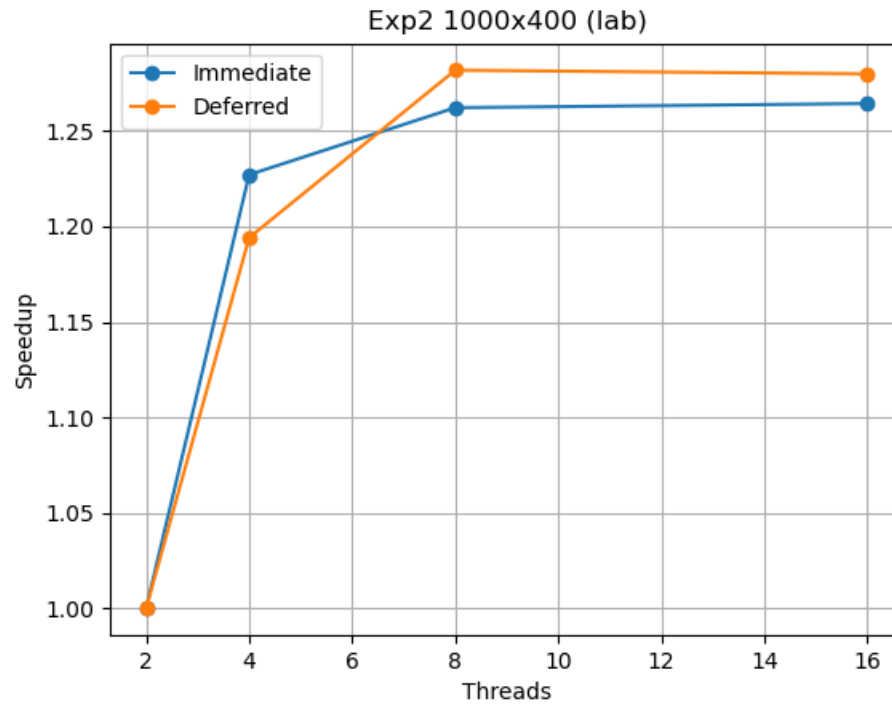


Figure 9: Experiment 2: Speedup vs threads for the 1000×400 grid on the Lab PC. The plot includes both Parallel Immediate and Parallel Deferred implementations.

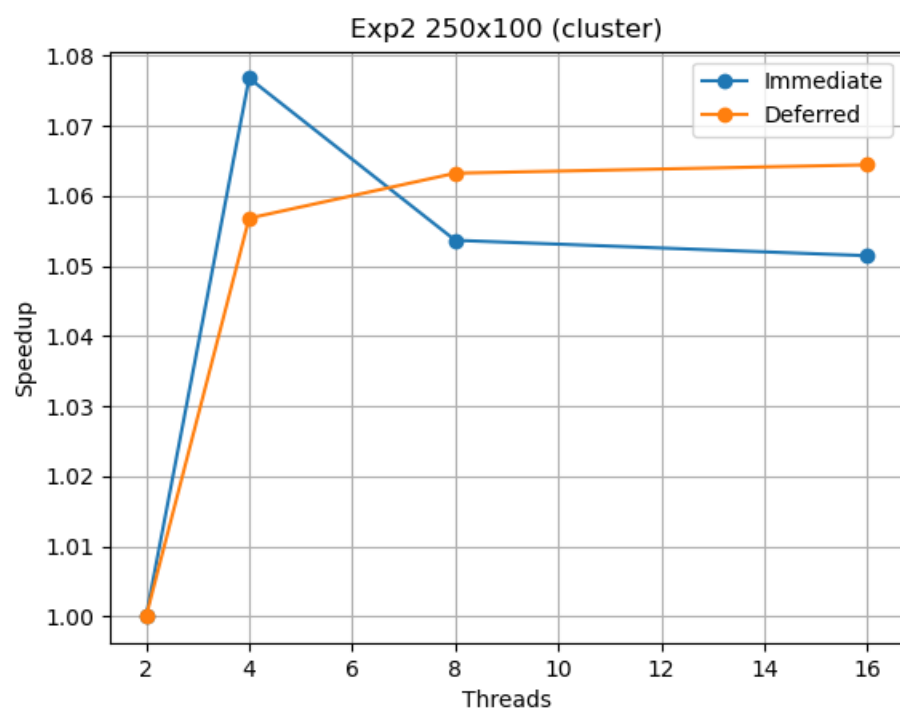


Figure 10: Experiment 2: Speedup vs threads for the 250×100 grid on the HPC cluster. The plot includes both Parallel Immediate and Parallel Deferred implementations.

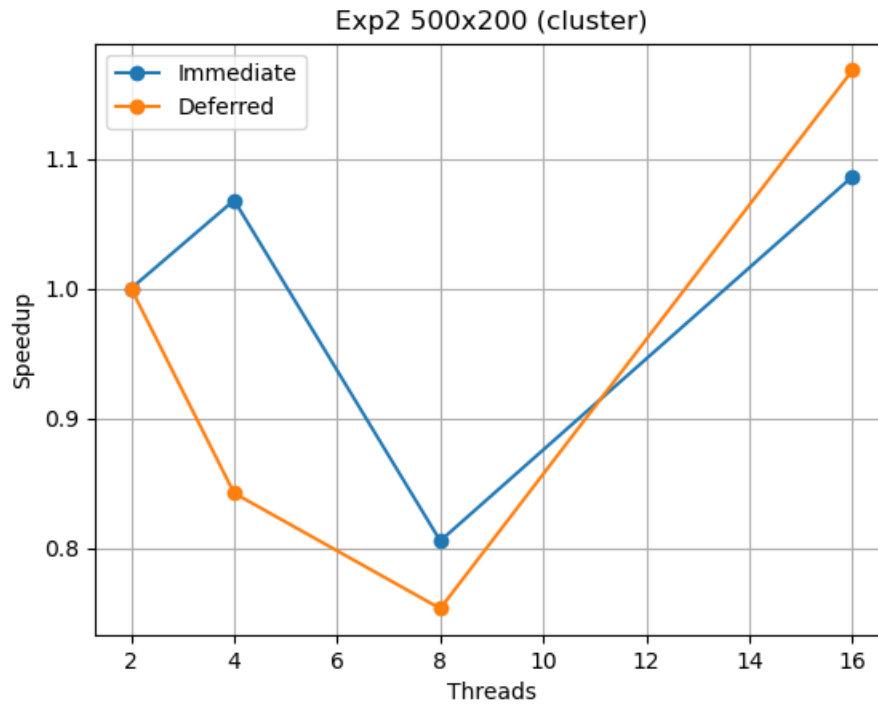


Figure 11: Experiment 2: Speedup vs threads for the 500×200 grid on the HPC cluster. The plot includes both Parallel Immediate and Parallel Deferred implementations.

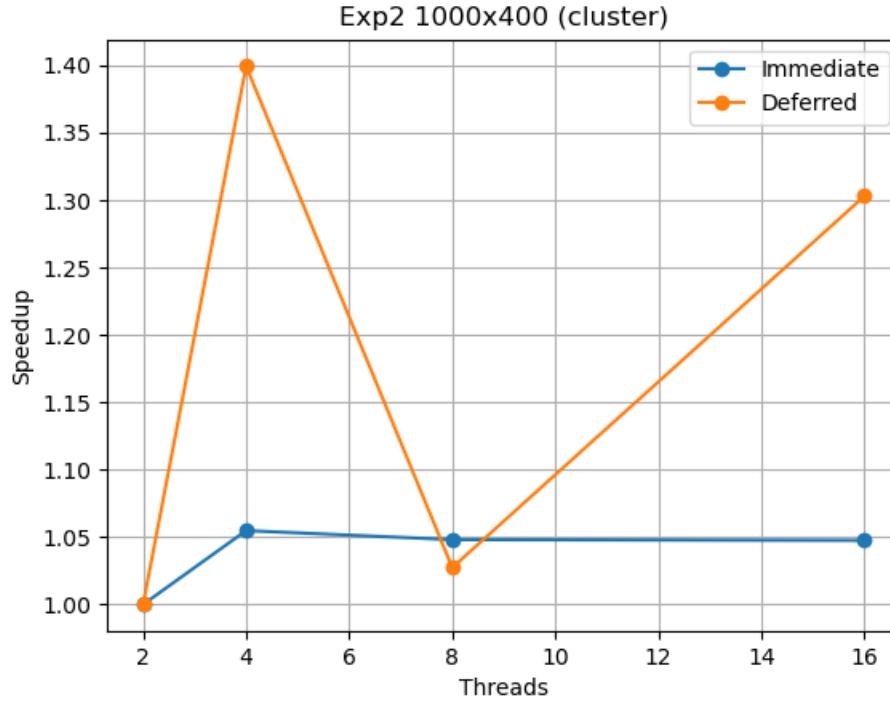


Figure 12: Experiment 2: Speedup vs threads for the 1000×400 grid on the HPC cluster. The plot includes both Parallel Immediate and Parallel Deferred implementations.

5.4 Comparison with Assignment 04 (Insertion vs No Insertion)

This comparison highlights the impact of particle insertion and deletion on parallel scalability. Parallel Immediate represents the baseline from Assignment 04 (no deletion), while Parallel Deferred represents the current implementation with insertion and deletion.

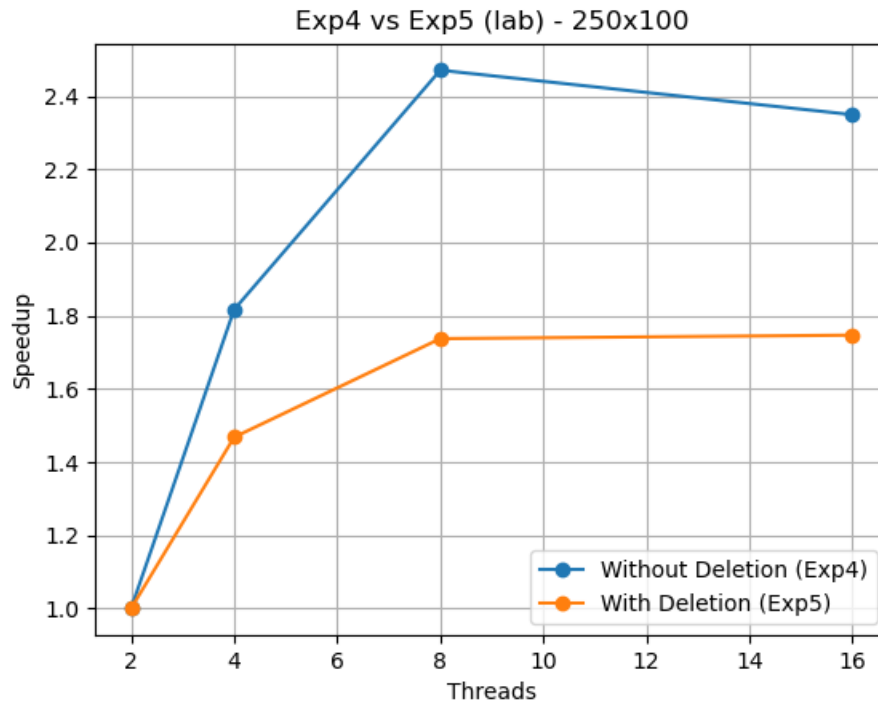


Figure 13: Comparison of speedup vs threads for the 250×100 grid on the Lab PC. Parallel Deferred shows reduced speedup due to compaction and insertion overhead.

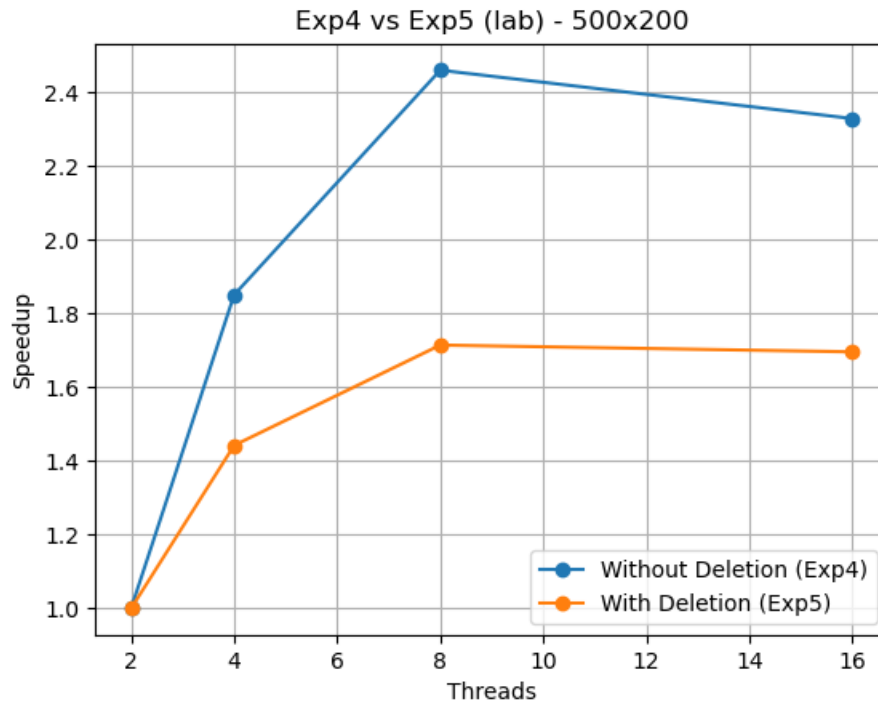


Figure 14: Comparison of speedup vs threads for the 500×200 grid on the Lab PC. The gap between Parallel Immediate and Parallel Deferred reflects additional memory operations.

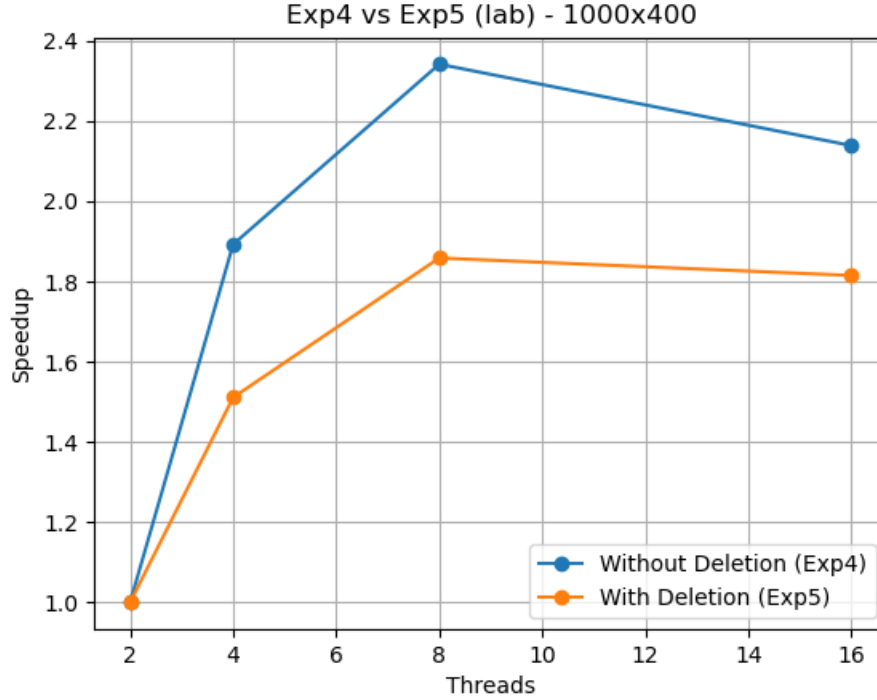


Figure 15: Comparison of speedup vs threads for the 1000×400 grid on the Lab PC. Larger workloads reduce relative overhead but differences remain visible.

5.5 Discussion of Observed Trends

The serial mover is expected to scale approximately linearly with particle count because each particle undergoes a constant amount of work. The larger grids increase memory footprint and can increase cache pressure, so the highest grid size typically shows a modest slowdown.

For the parallel mover, the speedup tends to improve as thread count increases from 2 to 8, but the gains can flatten at 16 threads because the computation becomes more memory-bound and the overhead of compaction and refill becomes more visible. The deferred version is usually slower than the immediate version because it performs extra work to compact valid particles and refill the invalid tail.

The Lab PC and HPC cluster may show different scaling behavior depending on core frequency, cache hierarchy, and memory bandwidth. The serial interpolation step often benefits from a faster per-core clock, while the mover can benefit from memory throughput and thread-level parallelism.

The comparison with Assignment 04 shows that the insertion and deletion logic introduces additional overhead, reducing parallel speedup. This is primarily due to compaction, memory movement, and synchronization costs in the deferred implementation.

The gap between the two curves increases with thread count, indicating that the overhead of insertion and deletion becomes more pronounced at higher parallelism.

In some cluster configurations, speedup drops below 1 due to thread overhead and memory contention dominating the computation.

The mover operation exhibits memory-bound characteristics, which limits ideal parallel scalability despite independent particle updates.

6 Analysis Tasks

Q1. Determine the theoretical time complexity of interpolation and mover operations.

Both interpolation and mover process each particle with a constant number of arithmetic operations and memory accesses. Therefore, for a single iteration, the time complexity is linear in the number of particles:

$$T_{\text{interp}}(N) = O(N), \quad T_{\text{mover}}(N) = O(N)$$

Since the simulation is executed for `Maxiter` iterations, the total runtime scales as:

$$T_{\text{total}} = O(N \cdot \text{Maxiter})$$

For the deferred mover, the compaction and refill steps are also linear, so the overall asymptotic complexity remains the same.

Q2. Explain why the serial mover scales nearly linearly with particle count in Experiment 1.

The mover applies the same constant amount of work to every particle: generate random displacements, update the position, check the boundaries, and either keep the particle or replace it. Because the per-particle work is fixed, doubling the particle count roughly doubles the runtime. Small deviations from perfect linearity arise from cache effects, random number generation overhead, and system noise.

Q3. Explain the parallel scalability behavior in Experiment 2.

The parallel mover scales well at moderate thread counts because particle updates are independent, so threads can process distinct particles without synchronization. However, the speedup does not continue to increase indefinitely. As thread count rises, the code becomes limited by memory bandwidth, cache contention, and the overhead of compaction and refill in the deferred version. The parallel immediate baseline (without insertion/deletion) is usually faster than parallel deferred because it avoids the extra rearrangement step.

Q4. Explain the trend of per-particle execution time versus PPC.

The per-particle execution time shows a non-monotonic trend due to the interplay of fixed overheads, cache behavior, and memory access patterns. At low PPC, overhead dominates, while at higher PPC, memory pressure and cache effects influence performance.

Q5. Explain the performance differences between the Lab PC and the HPC cluster.

The Lab PC may outperform the HPC cluster for some serial or lightly threaded cases because of its higher per-core clock frequency and lower latency. The HPC cluster may perform better for memory-intensive loops because of its higher memory bandwidth. The exact outcome depends on whether the workload is dominated by arithmetic, memory traffic, or thread overhead. The

deferred mover can especially expose differences in memory behavior because it performs additional compaction and refill operations.

Q6. Describe the particle–mesh workflow pictorially.

The workflow proceeds in a fixed order for every iteration:

Particles $\rightarrow$ Interpolation $\rightarrow$ Mover $\rightarrow$ Updated Particles

Interpolation transfers particle contributions to the mesh. The mover then updates particle positions, possibly deleting particles that leave the domain and generating new random particles to maintain the particle count. This pipeline is repeated for each iteration of the simulation.

Q7. Why does the deferred parallel mover need a more careful implementation?

A naive parallel compaction strategy that uses a shared write index would create a race condition because multiple threads could try to write to the same destination slot at the same time. The corrected implementation uses per-thread buffers and a small offset computation so that each thread writes to a unique region of the output array. This preserves correctness while maintaining parallel performance.

7 Conclusion

- The Assignment 05 particle–mesh workflow was extended to support particle insertion and deletion while preserving the total number of particles.
- The serial mover shows near-linear scaling with particle count, which matches the expected $O(N \cdot \text{Maxiter})$ behavior.
- The parallel mover provides substantial speedup because particle updates are independent, but the deferred version incurs additional overhead from compaction and refill.
- The Lab PC and HPC cluster show different performance characteristics because of differences in core frequency, cache hierarchy, and memory bandwidth.
- The required plots for scaling, PPC behavior, and parallel speedup should be inserted in the placeholder slots provided in this report.